

DEV2 – JAVL – Laboratoire Java**TD 11 – Interface****Résumé**

Dans ce TD, nous allons aborder la notion d'interface, ceci nous permettra de traiter certains objets de classes différentes comme des objets du même type.

Table des matières

1	Introduction	2
2	Exercices	2

1 Introduction

Les interfaces en java permettent de définir des contrats que les classes doivent respecter. Ce contrat a la forme d'une liste de méthodes.

L'intérêt des interfaces est de les utiliser comme type de variable/attribut/paramètre/-retour (p.ex. l'interface List). Alliées au polymorphisme, elles donnent plus de souplesse aux implémentations.

- ▷ Une interface est déclarée par le mot clef `interface`.
- ▷ Une interface définit un contrat sous la forme d'une liste de signatures de méthodes automatiquement publiques.
- ▷ Chaque méthode ne contient ni code ni bloc d'instruction et se termine par un point virgule.
- ▷ Les attributs sont automatiquement public et static.
- ▷ Une classe déclare implémenter une ou plusieurs interfaces avec le mot clef `implements`.

Dans l'exemple suivant l'interface `View` déclare 2 méthodes :

```
public interface View {  
    void makeTitle(String title);  
    void makeSubTitle(String subtitle);  
}
```

La classe `DecoratorView` implémente l'interface `View` :

```
public class DecoratorView implements View {  
    public void makeTitle(String title){  
        // some code here.  
    }  
    public void makeSubTitle(String subtitle){  
        // some other code here.  
    }  
}
```

Notez que les méthodes déclarées dans l'interface doivent être implémentées avec la visibilité `public`.

2 Exercices

Dans cette partie, vous allez apprendre à créer et implémenter des interfaces. Dans un premier temps, une seule interface est utilisée, ensuite, vous allez apprendre à implémenter plusieurs interfaces et enfin, vous apprendrez à étendre les interfaces.

Exercice 1

Interface View

Les classes qui suivent doivent être écrites dans un package adapté par exemple

`g12345.dev2.td13`. Écrivez l'interface `View` qui possède **quatre méthodes** qui ont pour objectif de mettre en forme des éléments sur la sortie standard. Les méthodes sont les suivantes :

- ▷ `makeTitle(String title)`, cette méthode permet d'afficher un titre.
- ▷ `makeSubTitle(String subtitle)`, cette méthode permet d'afficher un sous-titre.
- ▷ `makeItem(String sentence)`, cette méthode permet d'afficher un élément d'une liste.
- ▷ `makeParagraph(String paragraph)`, cette méthode permet d'afficher un paragraphe.

Exercice 2 View DecoratorView

Écrivez la classe `DecoratorView` qui implémente l'interface `View` :

```
public class DecoratorView implements View {...}
```

Ses méthodes affichent les éléments selon le format suivant :

- ▷ Les titres sont entourés d'un cadre.
- ▷ Les sous-titres sont soulignés.
- ▷ Les éléments de listes sont marqués par une tabulation suivie d'un astérisque.
- ▷ Les paragraphes sont simplement marqués d'un passage à la ligne.

Tester l'implémentation de DecoratorView

Dans la méthode `main`, écrivez un morceau de code pour tester chacune des méthodes de `DecoratorView`. Le résultat attendu doit ressembler à ceci :

```
|-----|  
|  Titre  |  
|-----|  
  
Sous-titre  
-----  
  
Ceci est un paragraphe  
    * Interface View  
    * Classe DecoratorView  
    * Classe ColorView  
    * Classe HTMLView
```

Exercice 3 View ColorView

Écrivez la classe `ColorView` qui implémente l'interface `View`. Ses méthodes affichent les éléments selon le format suivant :

- ▷ Les titres sont dans une couleur (ex : bleu clair).
- ▷ Les sous-titres sont dans une autre couleur (ex : bleu).
- ▷ Les éléments de listes sont marqués par une tabulation suivie d'un tiret et sont colorés (ex : vert).
- ▷ Les paragraphes sont dans la couleur par défaut.

Les couleurs s'obtiennent par un caractère spécial *invisible* : bleu `\u001B[34m`, cyan `\u001B[36m`, noir `\u001B[30m`, rouge `\u001B[31m`, blanc `\u001B[37m`, vert `\u001B[32m`, jaune `\u001B[33m`, magenta `\u001B[35m`, retour à la couleur par défaut `\u001B[39m`.

Tester l'implémentation de ColorView

Dans la méthode `main`, écrivez un morceau de code pour tester les méthodes de la classe `ColorView`. Le résultat attendu doit ressembler à ceci :

```
Titre
Sous-titre
Ceci est un paragraphe
- Interface View
- Classe DecoratorView
- Classe ColorView
- Classe HTMLView
```

Exercice 4

BlockView (implements)

Nous aimerions créer des vues pour lesquelles les débuts et les fins de listes sont marqués. Par exemple, si on pense au format HTML, les débuts et fins de liste sont marqués par les balises `<ul>` et `</ul>`.

Créez une nouvelle interface nommée `BlockView`. Cette interface possède deux méthodes :

- ▷ `void beginItemize()`, méthode qui permet de marquer le début d'une liste d'éléments.
- ▷ `void endItemize()`, méthode qui permet de marquer la fin d'une liste d'éléments.

Exercice 5

View HTMLView

Écrivez la classe `HTMLView`. Cette classe implémente les **deux** interfaces `View` et `BlockView`. Voici la description des méthodes de la classe `HTMLView` :

- ▷ Les titres sont entourés des balises `<h1>` et `</h1>`.
- ▷ Les sous-titres sont entourés des balises `<h2>` et `</h2>`.
- ▷ Les éléments d'une liste sont entourés des balises `<li>` et `</li>`, chaque élément est précédé d'une tabulation afin d'obtenir un code indenté.
- ▷ Les paragraphes sont entourés des balises `<p>` et `</p>`.
- ▷ Le début des éléments d'une liste est marqué par la balise `<ul>`.
- ▷ La fin des éléments d'une liste est marquée par la balise `</ul>`.

Tester l'implémentation de HTMLView

Dans la méthode `main`, écrivez un morceau de code pour tester les méthodes de la classe `HTMLView`. Le résultat attendu doit ressembler à ceci :

```
<h1>Titre</h1>
<h2>Sous-titre</h2>
<p>Ceci est un paragraphe</p>
<ul>
  <li>Interface View</li>
  <li>Classe DecoratorView</li>
  <li>Classe ColorView</li>
  <li>Classe HTMLView</li>
</ul>
```

Modifiez votre code afin de permettre d'insérer des listes d'éléments en tant qu'élément d'une liste. Pensez à conserver une indentation cohérente.

Exercice 6

BlockView (extends)

Nous aimerions que tout utilisateur de l'interface `BlockView`, implémente l'interface `View`. Pour cela, il est possible de dire que l'interface `BlockView` est une extension de l'interface `View` (*extends*). Modifiez votre code afin que ce soit le cas.

Notez que vous pouvez maintenant modifier `HTMLView` pour qu'elle ne déclare implémenter que `BlockView`, il n'est plus nécessaire de déclarer aussi `View`.